Binary & Hexadecimal

2.1.1

CS 224

Dr. Chris Archibald

Welcome back to first grade !

- How do we know the value of a number?

105624

- What does the 5 mean?
- What about the 0?

Value of 105624

- 1 x 4 = 4
- 10 x 2 = 20
- 100 x 6 = 600
- 1000 x 5 = 5000
- 10000 x 0 = 0
- 100000 x 1 = 100000

Add them all together 105624

Place determines value

- In Base 10 (decimal)
- Place 0 - 1s place
- Place 1 - 10s place
- Place 2 - 100s place
- Etc.



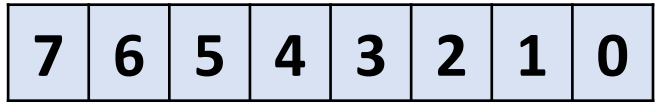
Places

- i-th place has value of 10^i
- This works with other bases as well
- i-th place has value of b^i (in base b)

Other bases

- We will use
  - base 2 (binary)
  - base 16 (hexadecimal)

Binary Place value

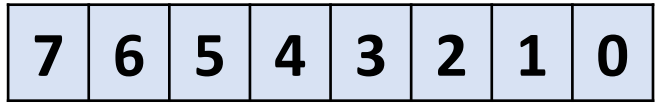


- Place 0 – 1
- Place 1 – 2
- Place 2 – 4
- Place 3 – 8
- Place 4 – 16
- Place 5 – 32
- Place 6 – 64
- Place 7 – 128
- Place 8 – 256

What is the value of?

- 1010011
  - 64 + 16 + 2 + 1 = 83
- 110
  - 4 + 2 = 6
- 10111
  - 16 + 4 + 2 + 1 = 23
- 0110101
  - 32 + 16 + 4 + 1 = 53

Hexadecimal Place value



- Place 0 – 1
- Place 1 – 16
- Place 2 – 256
- Place 3 – 4096
- Place 4 – 65,536

- Remember:
- A = 10, B = 11, C = 12, D=13, E = 14, F = 15

What is the value of?

- 12
  - (16 x 1) + 2 = 18
- 3a1
  - (256 x 3) + (16 x 10) + 1 = 929
- bf
  - (16 x 11) + 15 = 191
- c57
  - (256 x 12) + (16 x 5) + 7 = 3159

Hexadecimal and Binary

- Why do we use hexadecimal?
- Let's consider 4 bits (a nibble, or half of a byte)

Base 2	0000	0001	0010	0011	0100	0101	0110	0111
Base 10	0	1	2	3	4	5	6	7
Base 16	0	1	2	3	4	5	6	7
Base 2	1000	1001	1010	1011	1100	1101	1110	1111
Base 10	8	9	10	11	12	13	14	15
Base 16	8	9	A	B	C	D	E	F

Converting from Binary to Hexadecimal

- Every 4 bits becomes a hex digit
- 01001011101101010010?

0100	1011	1011	0101	0010
4	b	b	5	2

  - Becomes 0x4bb52
- 1110100100001

1	1101	0010	0001
1	d	2	1

- Becomes 0x1d21

Converting from Hexadecimal to Binary

- Every hex digit becomes 4 bits
- 51a4b
  - becomes 0101 0001 1010 0100 1011
- 42ff
  - becomes 0100 0010 1111 1111

Converting from Decimal to Binary

- To convert n to binary:
  - n%2 is least significant digit, then divide by 2
  - Repeat until n = 0
- Example: Convert 43 to binary
  - 43 % 2 = 1
  - 21 % 2 = 1
  - 10 % 2 = 0
  - 5 % 2 = 1
  - 2 % 2 = 0
  - 1 % 2 = 1
  - 0 % 2 = 0

Result = 0101011

Practice Problem 2.3 (solution page 144)

A single byte can be represented by 2 hexadecimal digits. Fill in the missing entries in the following table, giving the decimal, binary, and hexadecimal values of different byte patterns:

Decimal	Binary	Hexadecimal
0	0000 0000	0x00
167	_____	_____
62	_____	_____
188	_____	_____
_____	0011 0111	_____
_____	1000 1000	_____
_____	1111 0011	_____

Addition in other bases

- Works just the same as with base 10.
- Start with the ones, add the numbers, carry out to the next column

- Binary has few possibilities
  - 0 + 0 = 0 (carry 0)
  - 0 + 1 = 1 (carry 0)
  - 1 + 0 = 1 (carry 0)
  - 1 + 1 = 0 (carry 1)

- Adding 3 bits:
  - 0+0+0 = 0 (carry 0)
  - 0+0+1 = 1 (carry 0)
  - 0+1+1 = 0 (carry 1)
  - 1+1+1 = 1 (carry 1)

Binary addition

	1	0	1	1	0
+	1	1	0	1	1

Binary addition

				0	
	1	0	1	1	0
+	1	1	0	1	1
					1

Binary addition

			1	0	
	1	0	1	1	0
+	1	1	0	1	1
				0	1

Binary addition

		1	1	0	
	1	0	1	1	0
+	1	1	0	1	1
			0	0	1

Binary addition

	1	1	1	0	
	1	0	1	1	0
+	1	1	0	1	1
		0	0	0	1

Binary addition

1	1	1	1	0	
	1	0	1	1	0
+	1	1	0	1	1
	1	0	0	0	1

Binary addition

1	1	1	1	0	
	1	0	1	1	0
+	1	1	0	1	1
1	1	0	0	0	1

Practice Problem 2.4 (solution page 144)

Without converting the numbers to decimal or binary, try to solve the following arithmetic problems, giving the answers in hexadecimal. *Hint:* Just modify the methods you use for performing decimal addition and subtraction to use base 16.

A. 0x503c + 0x8 = _____

B. 0x503c − 0x40 = _____

C. 0x503c + 64 = _____

D. 0x50ea − 0x503c = _____

Multiplication

- Multiplying by the base
- In Base 10

- What is 572 * 10?
- How did you figure that out so quickly?
- Add a 0 on the right

Binary multiplication

- To multiply a binary number by 2, add a 0 on the right

1011 x 10 = 10110

This effectively shifts all the digits one to the left (increases place values)
You can do this using a **bit shift**

This is written in C as <<
Example: `a << 2` shifts bits of `a` two spaces to the left.
Empty spot gets a 0

Division

- How do we divide by the base?
  - Shift the digits of the number to the right!

- 562 / 10 = 56
  - Integer division, lose the remainder

- Binary?
- 1011 / 10 =
  - 0101
  - (Check: 11/2 = 5)

Bit shifts

- Left shift by *k* spaces = multiply by 2^k
  - `a << 2`
  - Multiply `a` by $2^2 = 4$
- Right shift by k spaces = divide by 2^k
  - `a >> 3`
  - Divide `a` by $2^3 = 8$

Converting to Binary . . . revisited

- The algorithm to convert *n* to binary:
  - Until *n = 0*:
    - next bit = *n % 2* *//produces bits from least significant to most*
    - *n = n / 2*
- What is going on?
- Let's walk through the algorithm at the bit level
  - *n* written out in binary
  - *n = 1011*
    - In decimal, *n = 11*
    - In hex, *n = b*

Converting to binary (*n%2; n=n/2;)*

- *n = 1011*
- *n % 2 = ?*
  - Is *n* even or odd? How can we tell this?
  - *n % 2* gives the least significant bit of *n*
  - *n % 2 = 1*
- *n = n / 2*
  - How can we divide by 2 in binary?
  - Right shift by 1 place!
  - *n = 0101*

Result = _ _ _ _

Converting to binary (*n%2; n=n/2;)*

- *n = 1011*
- *n % 2 = ?*
  - Is *n* even or odd? How can we tell this?
  - *n % 2* gives the least significant bit of *n*
  - *n % 2 = 1*
- *n = n / 2*
  - How can we divide by 2 in binary?
  - Right shift by 1 place!
  - *n = 0101*

Result = _ _ _ 1

Converting to binary (*n%2; n=n/2;)*

- *n = 0101*
- *n % 2 = 1*
- *n = n / 2*
  - *n = 0010*

Result = _ _ _ 1

Converting to binary (*n%2; n=n/2;)*

- *n = 0101*
- *n % 2 = 1*
- *n = n / 2*
  - *n = 0010*

Result = _ _ 1 1

Converting to binary (*n%2; n=n/2;)*

- *n = 0010*
- *n % 2 = 0*
- *n = n / 2*
  - *n = 0001*

Result = _ _ 1 1

Converting to binary (*n%2; n=n/2;)*

- *n = 0010*
- *n % 2 = 0*
- *n = n / 2*
  - *n = 0001*

Result = _ 0 1 1

Converting to binary (*n%2; n=n/2;)*

- *n = 0001*
- *n % 2 = 1*
- *n = n / 2*
  - *n = 0000*

Result = _ 0 1 1

Converting to binary (*n%2; n=n/2;)*

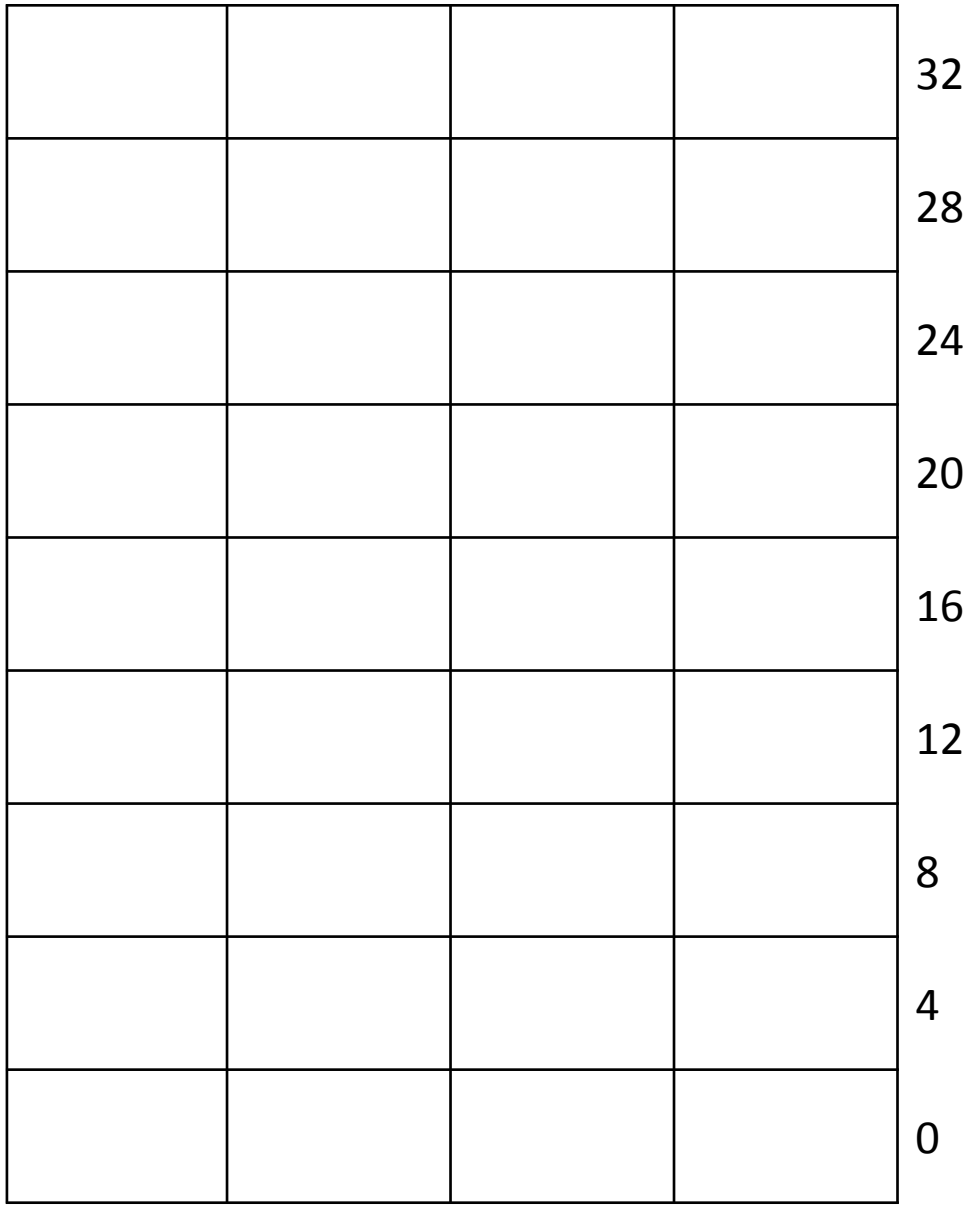
- *n = 0001*
- *n % 2 = 1*
- *n = n / 2*
  - *n = 0000*

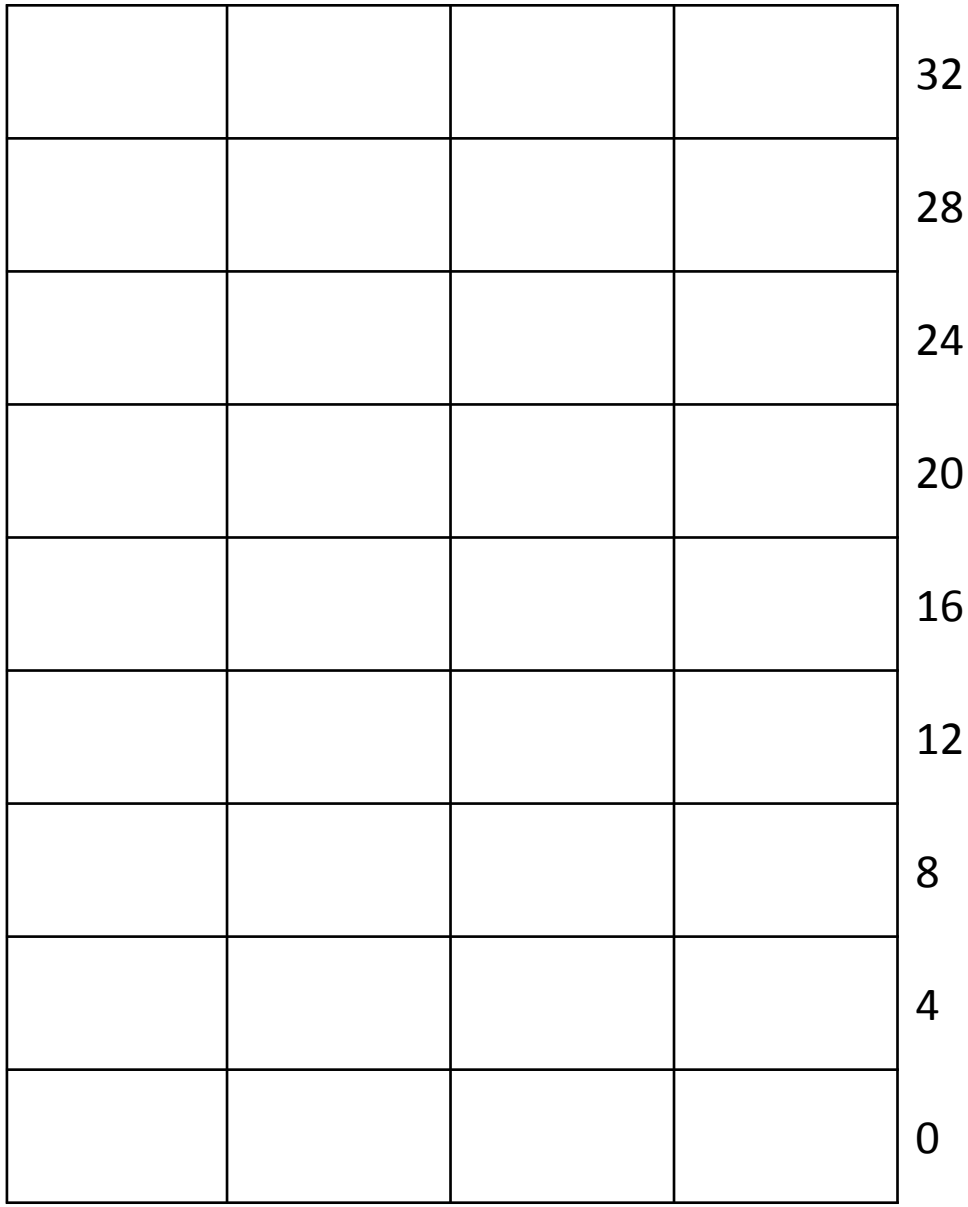
Result = 1 0 1 1

- Converting to binary just:
  - looking at the last bit (0/1 = even/odd = %2)
  - Shifting bits to right by one (= divide by 2)

Let’s do some coding practice problems . . .

- Examples of bit-shifting
- More playing with memory – similar to Lab 4
  - Scan in array of ints
  - Print out as string


32
28
24
20
16
12
8
4
0


32
28
24
20
16
12
8
4
0